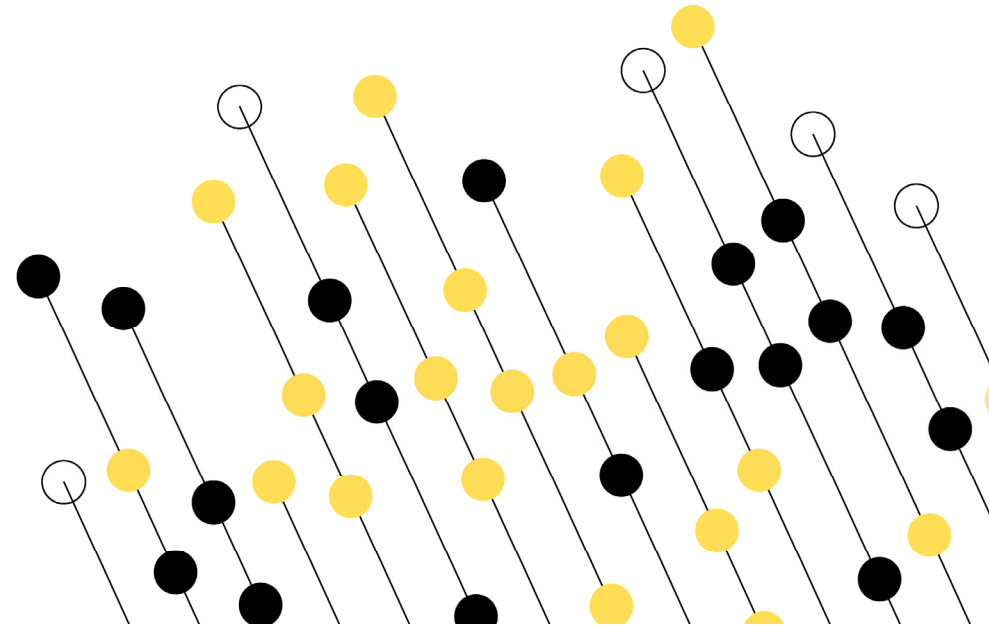


Object-oriented programming



Contents



Objectives

- To understand types and how to create objects.
- To grasp the concept of reference type behaviour.



Contents

- OO fundamentals — abstraction and encapsulation
- Defining reference types
- Creating objects (instances)



Hands-on labs

OO fundamentals — abstraction

- **Ability to represent a complex problem in simple terms:**
 - Creation of a high-level definition to simplify complexity.
 - Exposing only the relevant features. Hiding the complex implementation.
- **Stresses ideas, qualities, and properties — not particulars:**
 - Emphasises what an object is or does, rather than how it works.



Many drivers are unaware of car mechanics but can still drive a car.



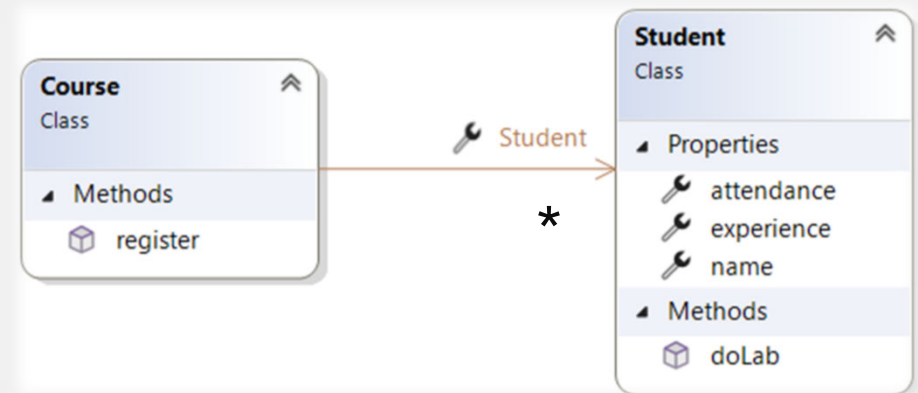
Interface



An example of using abstraction

- **Students (instances of type Student) attending a Course:**
 - Have '**attributes**' — name, experience, attendance record.
 - Have '**behaviour**' — register(), doLab().
- Are part of '**relationships**':
 - A student attends a course.
 - A course 'has' many students.
 - A Student can register on a course but doesn't need to know how it's done.

```
public void register(Student student) {  
    // code  
}
```



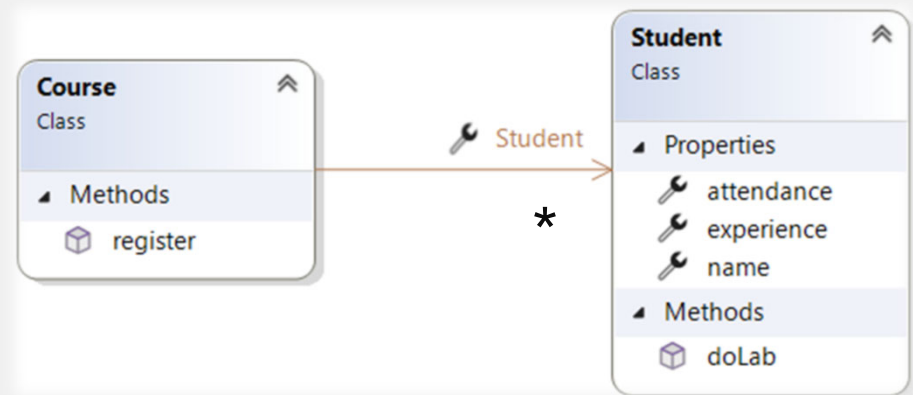
UML diagram

OO fundamentals — encapsulation

- Hiding object's implementation, making it self-sufficient



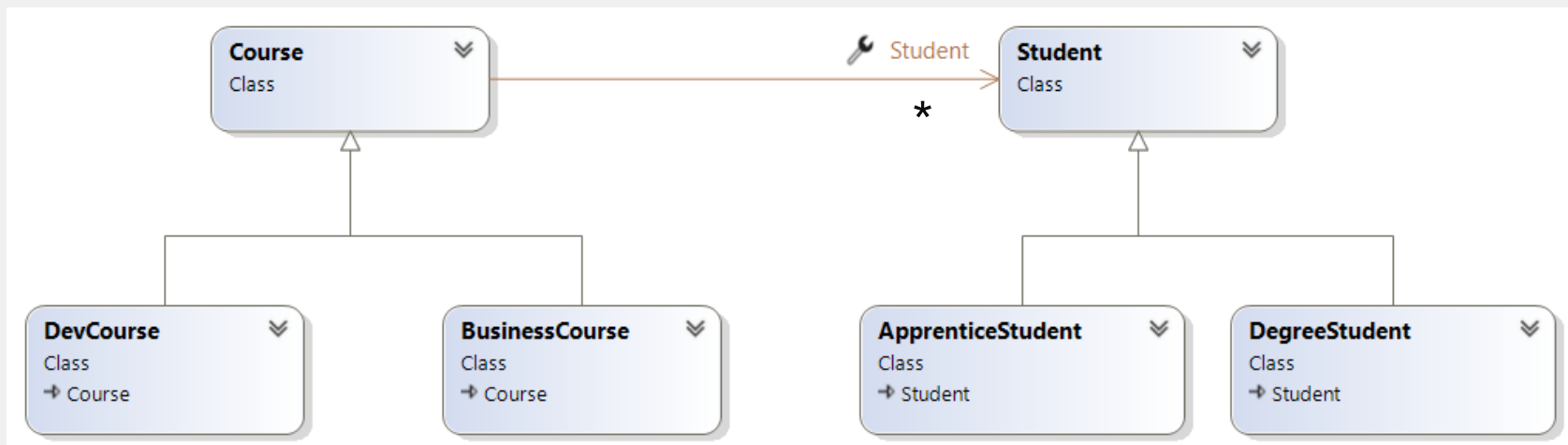
- **Process of enclosing code needed to do one thing well:**
 - Store necessary data in one object.
 - Build complexity from simple objects.
 - Hide internal details (e.g., lab performance).
 - Users just need to know the inputs and outputs.
 - Improve reliability, maintainability, and reuse.



Inheritance

Is an OOP concept where a subclass inherits properties and behaviour from a superclass.

It promotes **code reuse** and enables **polymorphism**.



OO fundamentals — objects communicate

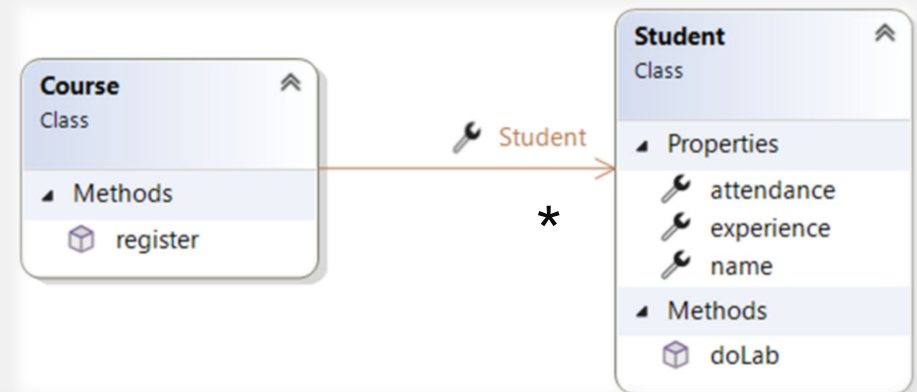
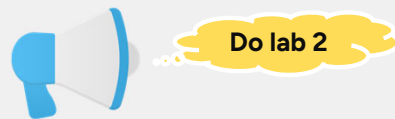
- Objects communicate via messaging (method calls).
 - Messages allow (receiving) object to determine implementation.



```
for(Student stu : students){
```

```
    stu.doLab(2);
```

```
}
```

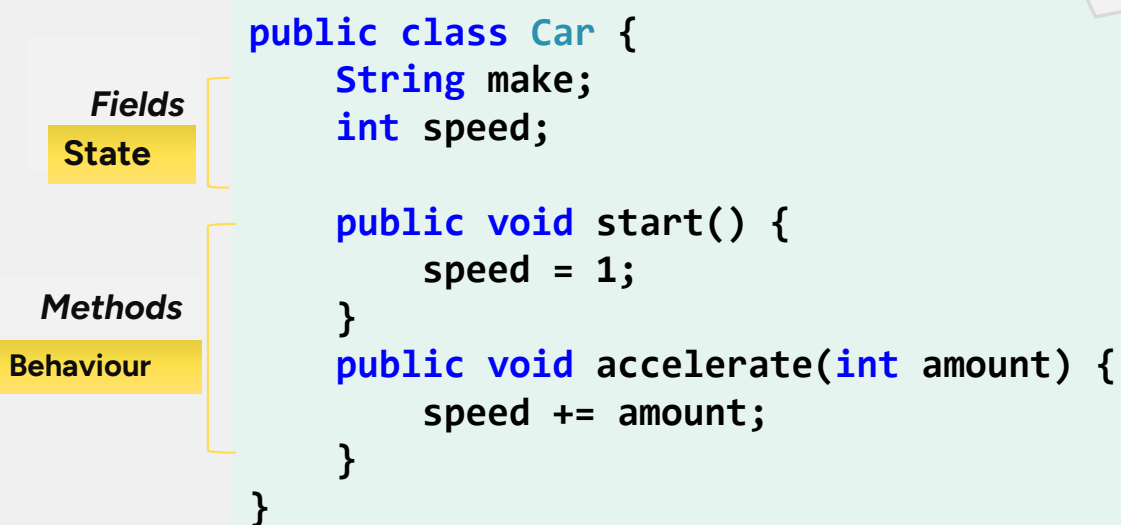


What is an OO data type?

Class definition is a blueprint for making objects:

- **Fields** — Data parts, **hold state**
- **Methods** — Methods, define **behaviour**

Save the file as **Car.java**
The class name must start
with an **uppercase** letter



```
public class Car {  
    String make;  
    int speed;  
  
    public void start() {  
        speed = 1;  
    }  
    public void accelerate(int amount) {  
        speed += amount;  
    }  
}
```

Public members are
an object's interface

Creating objects

- Objects are unique instances of a class with their own **state**.

Blueprint

```
public class Car {  
    String make;  
    int speed;  
  
    public void start() {  
        speed = 1;  
    }  
    public void accelerate(int amount) {  
        speed += amount;  
    }  
}
```

```
public static void main(...) {  
  
    Car car1 = new Car();  
    Car car2 = new Car();  
  
    car1.make = "Ford";  
    car2.make = "BMW";  
  
    car1.accelerate(10);  
    car2.accelerate(15);  
}
```

make: "Ford"
speed: 10

make: "BMW"
speed: 15

Instances of Car

Getters and setters

- Do not expose state

What is your
bank account
number?



My bank account
number is private!

Getters and setters (2)

- Do not expose state

```
public class Student {  
    private String name;  
    private int age;  
}
```

```
public static void main(...) {  
    Student stu = new Student();  
    stu.name = "Bob"; ✗  
    stu.age = 25; ✗  
}
```

Getters and setters (3)

- Do not expose state

```
public class Student {  
    private String name;  
    private int age;  
  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        if(name.length() > 1)  
            this.name = name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
    public void setAge(int age) {  
        this.age = age;  
    }  
}
```

```
public static void main(...) {  
  
    Student stu = new Student();  
  
    stu.name = "Bob"; ✗  
    stu.age = 25; ✗  
  
    stu.setName("Bob"); ✓  
  
    stu.setName("B"); ✓  
}
```

Name and age are private

But name is not changed

Parts of Java libraries do not consider state unless exposed as getters and setters.

Object construction

- Let's consider two classes and the two instances created.

```
public class Car {  
    private int speed;  
    private String make;  
}
```

```
Car myCar = new Car();
```

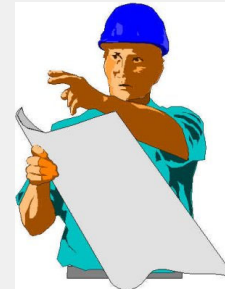
What make is this?
What is its speed?

```
public class Account {  
    private int id;  
    private String owner;  
}
```

```
Account myAccount = new Account();
```

What is the ID of this account?
Who owns it?

We need a constructor.



Constructor

```
public class Account {  
    private int id;  
    private String owner;  
  
    public Account (int id, String owner) {  
        this.id = id;  
        this.owner = owner;  
    }  
}
```

The same name as the class —
no return value, not even void.

Account myAccount = new Account(123, "Bob");



Account myAccount = new Account();

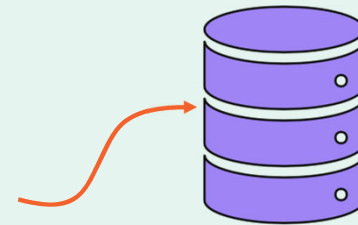


The default constructor does not exist. To create an Account,
you must provide the ID and the owner's name.

Object construction — overloading

- **Overloading provides alternative ways for creating an instance.**

```
public class Account {  
    private int id;  
    String owner;  
  
    public Account (int id, String owner) {  
        this.id = id;  
        this.owner = owner;  
    }  
  
    public Account (int id) {  
        this.id = id;  
        this.owner = getOwnerById(id);  
    }  
}
```



```
Account myAccount = new Account(123, "Bob");
```

```
Account myAccount = new Account(123);
```

Constructor chaining example

```
public class Car {  
    private String make;  
    private int numdoors;  
  
    ① public Car(String make, int nd ) {  
        this.make = make;  
        this.numdoors = nd;  
        // common code can sit here  
    }  
  
    ② public Car(String make) {  
        this(make, 4);  
    }  
}
```

Car car1 = new Car("BMW",5);
Car car2 = new Car("BMW");
Car car3 = new Car(); ❌

Chaining, using special syntax

The null reference

```
public Car getPoolCar() {  
    Car hireCar = null;  
  
    // attempt to get a Car  
    // from a pool of cars  
  
    return hireCar;  
}
```



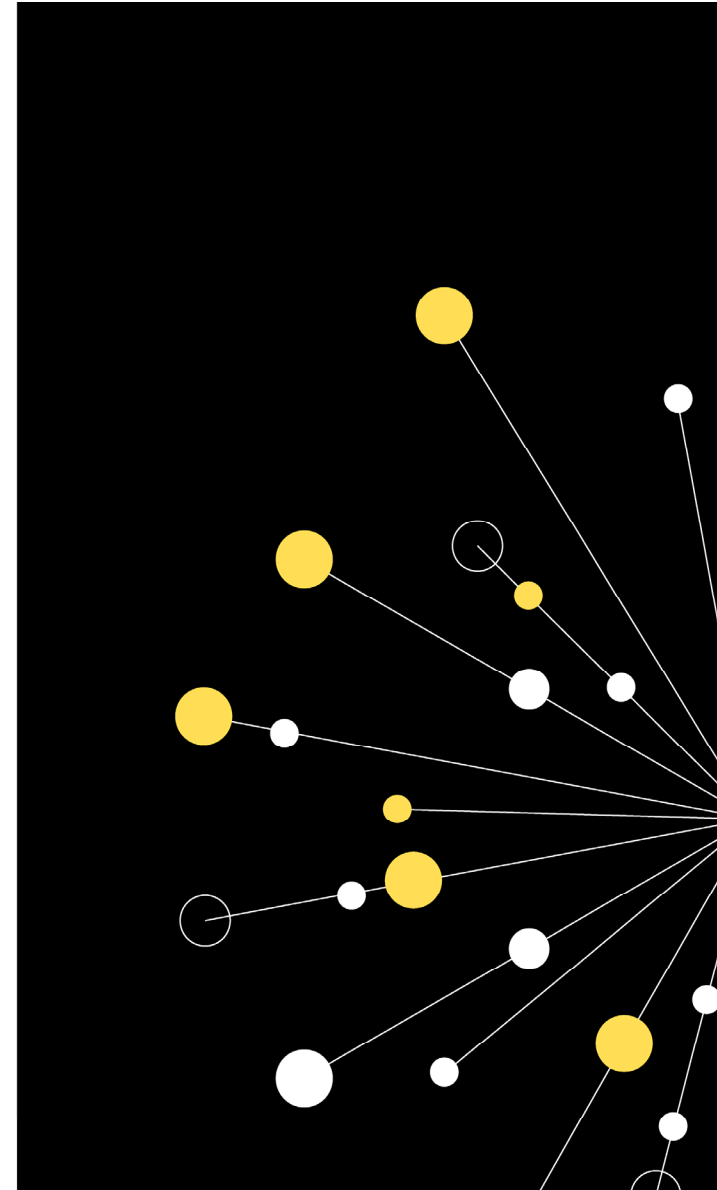
hireCar may not reference
an instance

```
public void hireCar() {  
    Car myCar = getPoolCar();  
  
    if (myCar != null) {  
        // Drive the car away  
    } else {  
        // No car available  
    }  
}
```

Can compare an object
reference with **null**

In this chapter, we reviewed...

- OO concept:
 - Abstraction and encapsulation.
- Defining types using the class keyword.
- Constructors.
- Getters and setters.
- Methods and object communication.
- Handling null reference.
- Record types.

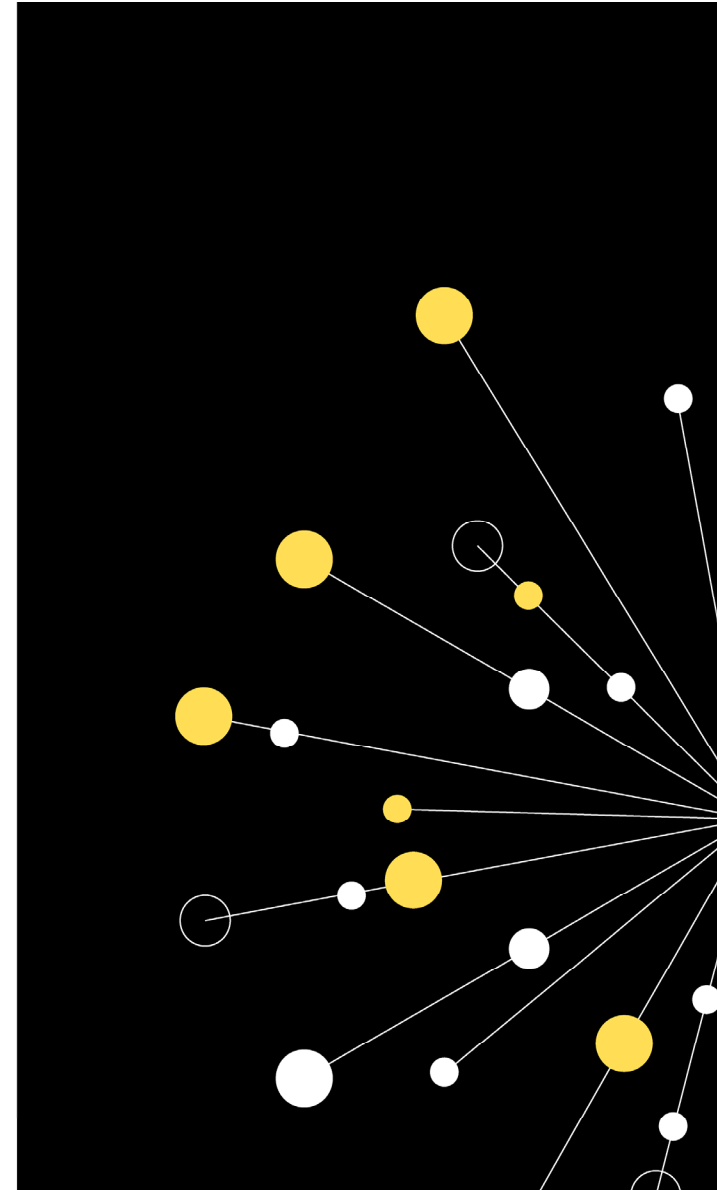


Lab

Objectives:

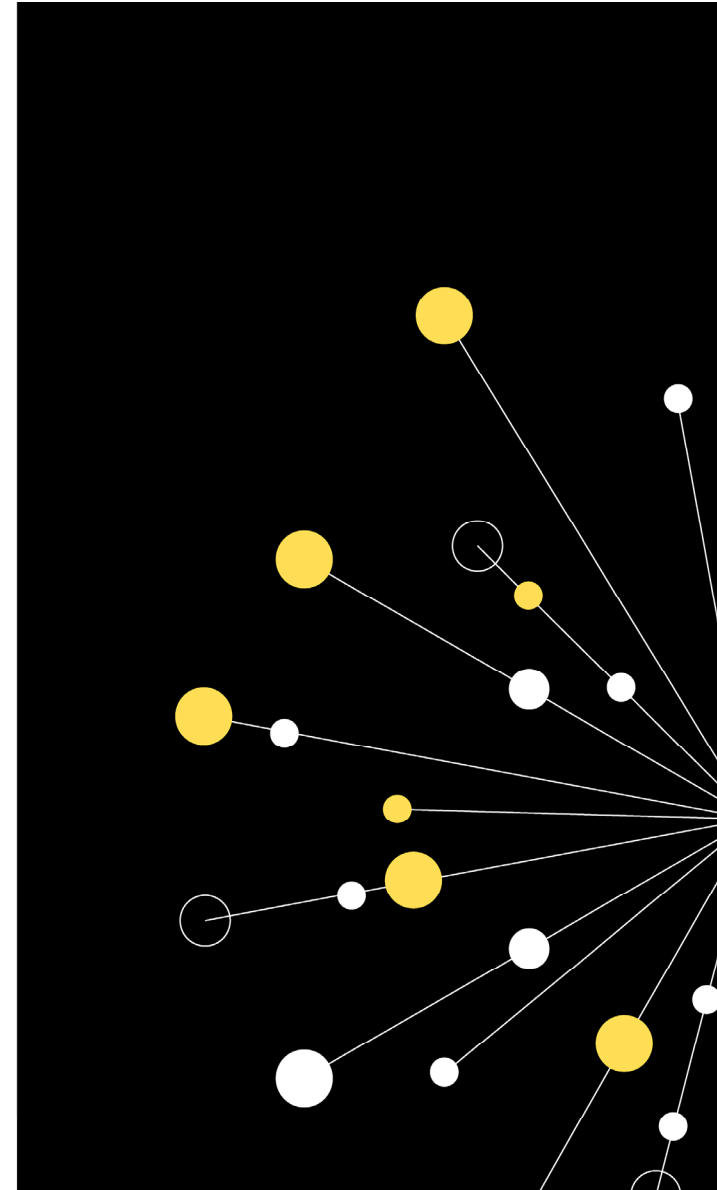
- Creating and using reference types.
- Passing reference types to a method.

Duration: 90 minutes.



Extra material

QA



Record types — Java 14+

```
record Student(String name, int age) { }
```

```
public class Main {  
    public static void main(String[] args)  
    {  
        Student s1 = new Student("Alice", 22);  
        System.out.println(s1);  
        System.out.println("Name: " + s1.name()); // default Getter  
        System.out.println("Age: " + s1.age());  
    }  
}
```

Default
toString()

```
Student[name=Alice, age=22]  
Name: Alice  
Age: 22
```

Records are immutable

Their fields are final and **cannot be changed** after the object is created (**no setter methods**).

They have simplified constructor and getters.

Record as method return value

```
record Student(String name, int age) { }

class Main {
    public static void main(String[] args)
    {
        Student student = getStudent();
        System.out.println("Student: " + student);
    }

    public static Student getStudent()
    {
        return new Student("Alice", 22);
    }
}
```

Student[name=Alice, age=22]

Record custom constructors

```
record Student(String name, int age) {  
  
    public Student {  
        if (age < 0) {  
            throw new IllegalArgumentException("Age cannot be negative!");  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student("Alice", -1);  
    }  
}
```

Exception in thread 'main' java.lang.IllegalArgumentException: Age cannot be negative!

Record default constructor

```
record Student(String name, int age)
{
    public Student()
    {
        this("Unknown", 0); // Default values
    }
}

public class Main {
    public static void main(String[] args)
    {
        Student s1 = new Student();
        System.out.println(s1);
        System.out.println("Name: " + s1.name());
        System.out.println("Age: " + s1.age());
    }
}
```

```
Student[name=Unknown, age=0]
Name: Unknown
Age: 0
```


Wrapper classes

- You have used the primitives — `int`, `long`, `double`, etc.
- You will know by now that **Object** is the 'top of the class hierarchy':
 - Can you supply a 'primitive' (`int` say) to a method expecting `Object`?
 - No, but you can supply an `Integer` reference.
- **Java provides `Integer`, `Long`, `Double`, etc., as wrapper classes:**
 - These exist to wrap the primitive in an object.
 - The compiler does the wrapping for you, known as 'auto-boxing'.

```
int x = 5;
Integer i = new Integer(x);    // not necessary
Integer i = x;                 // sufficient or just ..
expectsObjectReference(x);    // often seen
```

- Compiler can also do the 'un-boxing' for you.

```
int x = methodThatReturnsInteger(); // un-boxing
```

Wrapper class methods and constants

- What is 2 to the power 31 minus 1? The largest value an `int` can be:
 - 2147483647, but in Java it is the constant `Integer.MAX_VALUE`.
- Example of some methods of class `Integer` in use:

```
int x = 5;
Integer i = new Integer(x);           // create Integer object
String s = i.toString();              // (i + "") is easier!
s = "234"+"567";
Integer p = Integer.parseInt(s);      // or of course just ..
int y = Integer.parseInt(s);
Integer q = Integer.valueOf(x);       // alternative to constructor
int z = q.intValue();                 // but int z = q works
```

- These classes offer lots of methods for:
 - Bitwise manipulation, working in or converting to / from binary, octal, hexadecimal, etc.
- All objects of these wrapper class types like `String` are immutable.